Evaluating Transformer-Based Models for English-to-Python Code Generation on the MBPP Dataset

Dixin Zhang
MSDS 458
Northwestern University
Instructor Syamala Srinivasan
May 17, 2026

Final Research Report

Abstract

This study evaluates whether transformer-based models can generate syntactically valid Python code from natural language prompts using the Mostly Basic Python Problems (MBPP) dataset. The practical business question is whether chatbot-assisted programming tools are feasible for low-risk, entry-level coding support. Ten experiments were compared across scratch transformers, zero-shot CodeT5 models, and fine-tuned CodeT5 models. Model quality was evaluated using BLEU, Python syntax validity through ast.parse, training time, and inference time. Final model selection used validation BLEU to avoid tuning on the test set. Scratch models failed to learn usable code generation from the small MBPP split. Zero-shot CodeT5 produced parseable Python more often, but its outputs did not align closely with reference solutions. Fine-tuning produced the strongest results. C3, a five-epoch fine-tuned CodeT5-small model with greedy decoding, achieved the highest validation BLEU and a test BLEU of 4.61 with 46.0% syntax validity. C4 achieved the highest syntax validity at 60.0%. The findings support a controlled pilot, not autonomous production deployment, with syntax checks, sandboxed unit tests, human review, and ongoing monitoring.

Keywords: code generation, transformer, CodeT5, MBPP, BLEU, syntax validity

Introduction

This research evaluates the technical feasibility of using chatbot-style artificial intelligence systems to solve entry-level Python programming problems. In business settings, this capability could support junior developers, analysts, data scientists, and nontechnical employees who need small utility functions for routine work. However, generated code also creates operational risk when it is syntactically invalid, semantically incorrect, insecure, or difficult to maintain. Before considering broad enterprise adoption, management needs evidence about whether these systems can reliably translate natural language requirements into usable code.

The task is framed as English-to-Python sequence generation: each input is a natural language prompt, and each output is a Python solution. The central research question is whether transformer-based models can generate code with enough accuracy and syntax validity to justify further business evaluation. To answer this question, the study compares scratch-trained transformers, zero-shot pretrained CodeT5 models, and fine-tuned CodeT5 models using the same MBPP splits and evaluation metrics.

Literature Review

Transformer models are the foundation for modern sequence generation. Vaswani et al. (2017) introduced the attention-based Transformer architecture, which uses self-attention, cross-attention, positional encoding, and feed-forward layers to map input sequences to output sequences. This architecture is well suited to code generation because the model must connect natural language intent with structured Python syntax.

Program synthesis research provides the benchmark context for this study. Austin et al. (2021) introduced MBPP as a collection of 974 entry-level Python programming tasks with descriptions, reference solutions, and tests. Because MBPP tasks are designed to be solvable by beginning programmers, the dataset is appropriate for evaluating whether a model can support low-risk programming assistance.

Evaluation must consider both textual similarity and executable quality. BLEU, proposed by Papineni et al. (2002), measures n-gram overlap between generated and reference outputs and is useful when the task is treated as translation. However, code must also parse and run. Chen et al. (2021) emphasized functional correctness for code models through test-based evaluation, while this study uses Python syntax validity through ast.parse as a lightweight but necessary quality check.

Pretraining is especially important for small datasets. Goodfellow et al. (2016) note that deep learning models generally need substantial data and appropriate inductive bias. CodeT5, proposed by Wang et al. (2021), is an encoder-decoder transformer pretrained for code understanding and generation, making it a strong candidate for natural-language-to-code generation when task-specific data is limited.

Methods

Research Design and Model Topologies

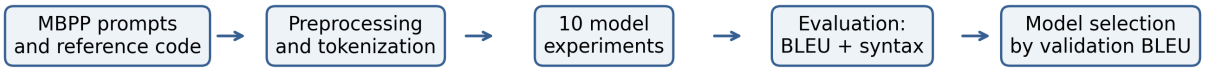
This study used an experimental machine learning design. Ten official experiments were conducted across three model families: scratch transformer models, zero-shot pretrained CodeT5 models, and fine-tuned CodeT5 models. The scratch experiments tested whether MBPP alone was sufficient for training from the ground up. The zero-shot experiments tested whether pretrained CodeT5 could solve the task without task-specific training. The fine-tuned experiments tested whether adapting CodeT5 to MBPP improved BLEU and syntax validity.

The final model-selection rule used validation BLEU rather than test BLEU. This choice follows standard machine learning practice: validation data is used for model selection and hyperparameter comparison, while test data is reserved for final reporting.

Figure 1

Research Pipeline for the MBPP Code Generation Study

Note. The workflow begins with MBPP prompts and reference code, then proceeds through preprocessing, model experimentation, evaluation, and validation-based model selection.



Data, Implementation, and Metrics

The dataset was MBPP, a benchmark of approximately 1,000 crowd-sourced Python programming problems designed for entry-level programmers (Austin et al., 2021; Google Research, n.d.). The processed experimental splits used in this assignment contained 374 training examples, 90 validation examples, and 50 test examples. Each example included a natural language task description and a reference Python solution.

The project was implemented in Python. Scratch transformer experiments were run in Google Colab and locally in VS Code with PyTorch/CUDA when cloud execution was slow. Pretrained and fine-tuned CodeT5 experiments were run in Colab and integrated into the final summary notebook. Completed experiment outputs were reused to avoid unnecessary retraining.

The main accuracy metric was BLEU, reported on a 0-100 scale. Syntax validity was calculated as the percentage of generated outputs that could be parsed as Python code using ast.parse. Training time and average inference time were also recorded because model speed and operating cost matter for business adoption.

Results

Table 1

Decision-Relevant Performance Summary Across the Ten Official Experiments

Note. BLEU is reported on a 0-100 scale. Syntax validity is the percentage of generated outputs that parsed as valid Python. The table focuses on validation and test metrics used for model selection and practical decision-making.

Exp.	Topology	Model	Dec.	Val BLEU	Test BLEU	Val Syn.%	Test Syn.%	Train Time (sec)	Avg Test Inf. (sec)
A1	Scratch Transformer	Small Scratch Transformer	greedy	0.000	0.000	0.0	0.0	13.75	1.901
A2	Scratch Transformer	Larger d_model Scratch Transformer	greedy	0.000	0.000	0.0	0.0	11.83	2.626
A3	Scratch Transformer	More Dropout Scratch Transformer	greedy	0.175	0.569	2.2	2.0	3.43	0.736
B1	Zero-shot	CodeT5-small	greedy	0.012	0.000	37.8	38.0	0.00	0.069
B2	Zero-shot	CodeT5-small	beam=3	0.017	0.000	37.8	42.0	0.00	0.098
B3	Zero-shot	CodeT5-small	beam=3	0.000	0.000	12.2	24.0	0.00	0.110
C1	Fine-tuned	CodeT5-small	greedy	3.553	3.732	35.6	38.0	47.45	0.434
C2	Fine-tuned	CodeT5-small	greedy	1.558	1.873	8.9	16.0	46.29	0.451
C3	Fine-tuned	CodeT5-small	greedy	5.031	4.606	36.7	46.0	77.86	0.466
C4	Fine-tuned	CodeT5-small	beam=3	4.476	4.453	46.7	60.0	47.45	0.678

Model-selection snapshot: C3 is the official selected model because it has the highest validation BLEU. C4 is the strongest alternative when syntax validity is the main priority.

Best Model Selection

The best model was selected by validation BLEU, which identified C3 as the strongest official model. C3 was a fine-tuned CodeT5-small model trained for five epochs with a learning rate of 5e-5 and greedy decoding. It achieved the highest validation BLEU of 5.031 and a test BLEU of 4.606, with 46.0% test syntax validity. C4 achieved the highest test syntax validity at 60.0% and a nearly comparable test BLEU of 4.453. Therefore, C3 is the best model under the formal selection rule, while C4 is a practical alternative if management values parseability more than BLEU.

Figure 2

Test BLEU Comparison Across Experiments

Note. Fine-tuned CodeT5 experiments produced the highest test BLEU values, while scratch and zero-shot models had near-zero test BLEU.

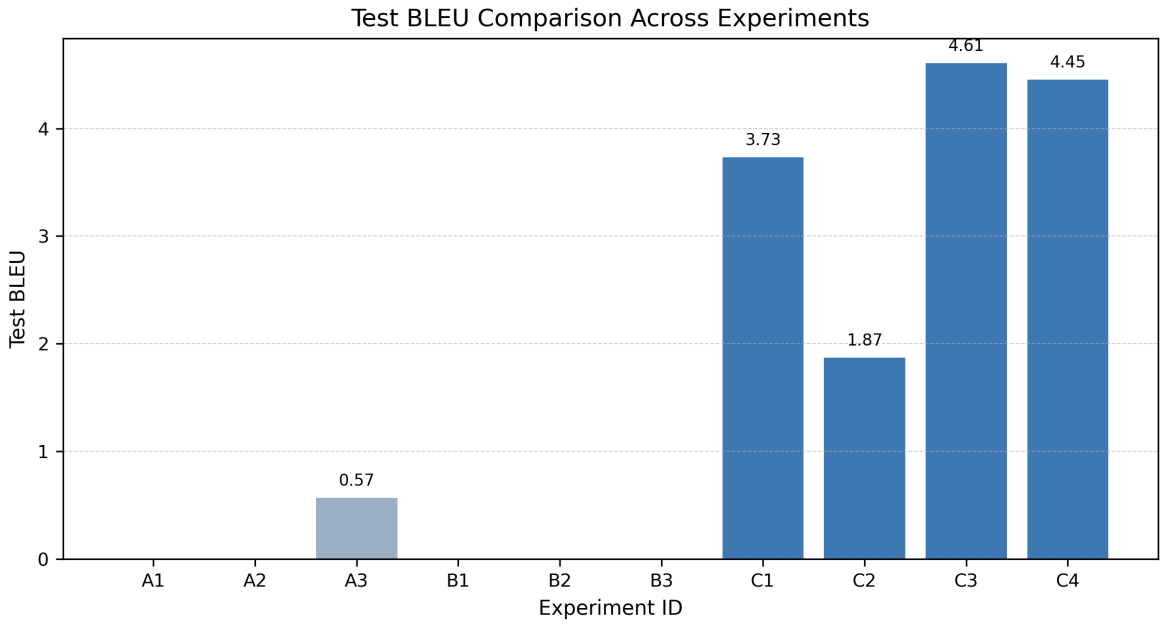
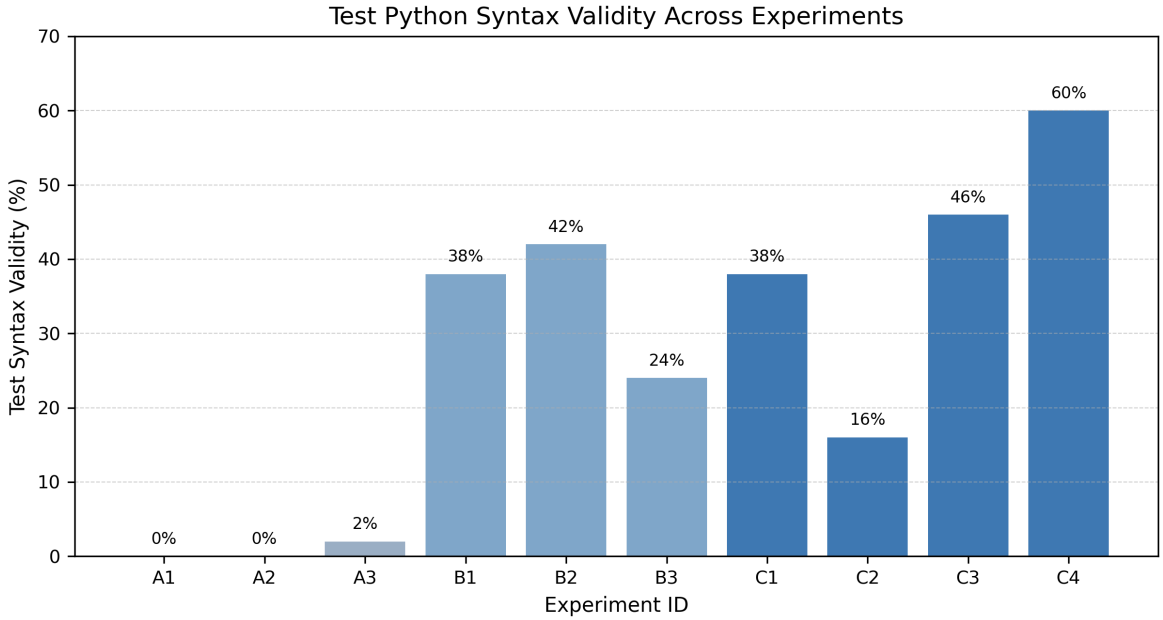


Figure 3

Test Python Syntax Validity Across Experiments

Note. C4 achieved the highest syntax validity. Zero-shot CodeT5 generated syntactically valid code more often than scratch transformers despite low BLEU.



Model Family Findings

The scratch transformer models performed poorly. A1 and A2 produced zero BLEU and zero syntax validity on the test set, while A3 improved only slightly. This result is expected because a few hundred training examples are not enough for a transformer to learn natural language structure, Python syntax, and task-specific mappings from scratch.

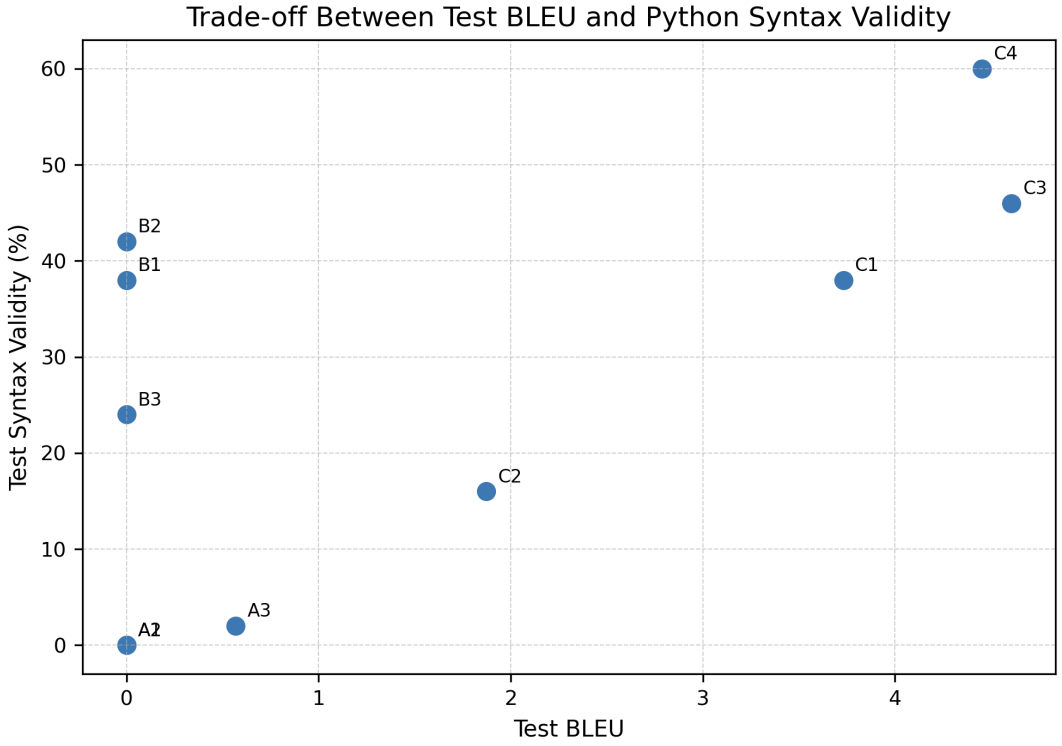
Zero-shot CodeT5 produced more syntactically valid code than the scratch models, but its BLEU scores were near zero. This suggests that CodeT5 had learned general code structure during pretraining, but without MBPP fine-tuning it did not reliably generate solutions aligned with the reference code.

Fine-tuned CodeT5 produced the strongest results. C3 had the best validation and test BLEU, while C4 had the best syntax validity. The comparison indicates that fine-tuning improved reference similarity and that beam search can improve syntactic correctness in the fine-tuned setting.

Figure 4

Trade-off Between Test BLEU and Python Syntax Validity

Note. BLEU and syntax validity measure different quality dimensions. A model can generate parseable code without closely matching the reference solution.



Training Behavior

A3 training curves show that local scratch transformer training improved token-level behavior: training and validation loss decreased, and masked token accuracy increased across epochs. However, this did not translate into strong code generation quality. The gap shows that token-level training loss alone is not sufficient for evaluating generated programs.

Figure 5

A3 Scratch Transformer Training and Validation Loss

Note. Loss decreased during training, but final generated code quality remained limited.

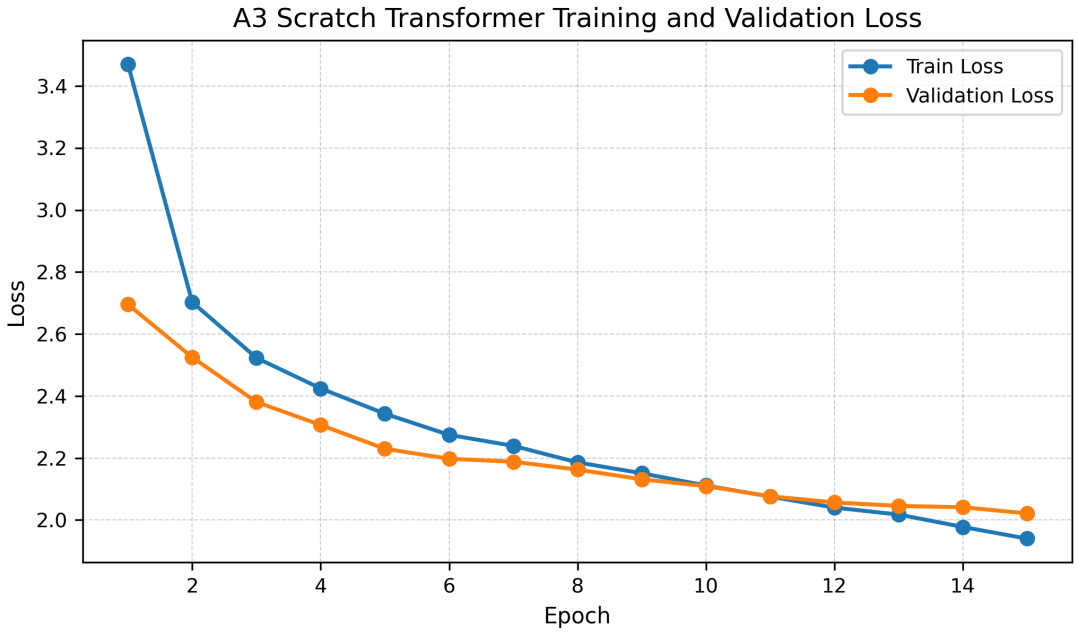
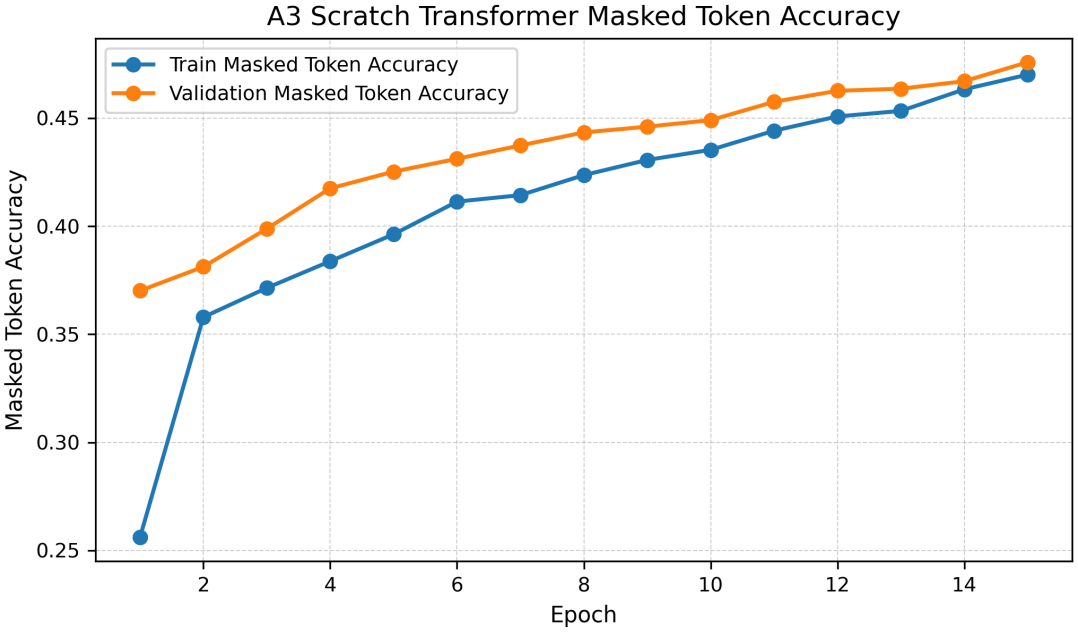


Figure 6

A3 Scratch Transformer Masked Token Accuracy

Note. Masked token accuracy increased across epochs, but syntax validity and BLEU remained low on the test set.



Conclusions and Management Recommendations

This study shows that chatbot-assisted code generation for entry-level Python tasks is technically feasible under controlled conditions. Fine-tuned pretrained models performed

meaningfully better than scratch-trained and zero-shot models. Therefore, a company should not rely on a small internally trained model built from limited examples. A pretrained code model fine-tuned on company-relevant tasks is the more practical path.

The recommended model from this experiment is C3 because it had the highest validation BLEU and the best test BLEU. However, C4 deserves further consideration if syntactic correctness is prioritized. The practical strategy is to use a fine-tuned model with beam search or constrained decoding when parseability and review efficiency matter more than speed.

The results should be treated as an initial feasibility study rather than evidence for autonomous deployment. BLEU scores were still low in absolute terms, and the best syntax validity did not exceed 60%. Generated code therefore requires validation before production use.

Management should adopt a staged pilot. First, limit use to low-risk tasks such as list manipulation, string processing, utility functions, and data-cleaning snippets. Second, require automated syntax checks before output is shown to users. Third, execute unit tests in a sandbox environment and add Pass@k or unit-test pass rate in the next evaluation phase. Fourth, require human review before code is merged into production repositories. Fifth, log prompts, outputs, syntax results, tests, and user corrections for future fine-tuning. Finally, security controls so generated code cannot access credentials, production databases, local file systems, or external networks without review.

Overall, AI-assisted code generation is promising but not ready for unrestricted use. Fine-tuned pretrained models provide the best technical foundation, but business adoption should depend on a broader evaluation framework that includes syntax validity, unit-test performance, security, maintainability, inference cost, user satisfaction, and developer productivity.

References

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., & Sutton, C. (2021). Program synthesis with large language models. arXiv. https://arxiv.org/abs/2108.07732

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., ... Zaremba, W. (2021). Evaluating large language models trained on code. arXiv. https://arxiv.org/abs/2107.03374

Chen, T. (n.d.). Research: A Vision Transformer Machine Learning Model for COVID-19 Diagnosis Using Chest X-Ray Images [GitHub repository]. GitHub. https://github.com/TyBruceChen/Research-A-Vision-Transformer-Machine-Learning-Model-for-COVID-19-Dagnosis-Using-Chest-X-Ray-Images

Chen, T., Philippi, I., Phan, Q. B., Nguyen, L., Bui, N. T., daCunha, C., & Nguyen, T. T. (2024). A vision transformer machine learning model for COVID-19 diagnosis using chest X-ray images. Healthcare Analytics, 5, Article 100332. https://www.sciencedirect.com/science/article/pii/S2772442524000340

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.

Google Research. (n.d.). Mostly Basic Python Problems Dataset. GitHub. https://github.com/google-research/google-research/tree/master/mbpp

Jurafsky, D., & Martin, J. H. (2025). Speech and language processing. Draft edition. https://web.stanford.edu/~jurafsky/slp3/

Papineni, K., Roukos, S., Ward, T., & Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, 311-318. https://aclanthology.org/P02-1040/

Python Software Foundation. (n.d.). ast - Abstract syntax trees. Python documentation. https://docs.python.org/3/library/ast.html

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. arXiv. https://arxiv.org/abs/1706.03762

Wang, Y., Wang, W., Joty, S., & Hoi, S. C. H. (2021). CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, 8696-8708. https://aclanthology.org/2021.emnlp-main.685/